

DSC 630 - Data Mining

Week 6 Assignment

6.2 Assignment: Milestone-3 Preliminary Analysis

Manoj K Kola

Milestone-2 Summary

Introduction

Customer churn is a critical challenge in the telecommunications industry, where companies must continuously retain customers amid intense competition. The ability to predict whether a customer is likely to leave a service provider allows organizations to take proactive measures such as targeted marketing, personalized offers, and improved customer service.

This project focuses on building predictive models to identify customers at risk of churning using the publicly available **Telco Customer Churn Dataset**.

Dataset Source: [Kaggle – Telco Customer Churn](#)

Dataset Overview

The Telco Customer Churn dataset includes the following features:

Category	Features
Demographics	Gender, SeniorCitizen, Partner, Dependents
Account Information	Tenure, Contract Type, Payment Method
Services	Internet Service, Online Security, Streaming TV/Movies
Billing	MonthlyCharges, TotalCharges
Target Variable	Churn (Yes / No)

Proposed Models & Rationale

Model	Rationale
Logistic Regression	Baseline model; interpretable; benchmarks initial performance
Decision Tree	Captures non-linear relationships; visualizable decision rules

Model	Rationale
Random Forest	Reduces overfitting; handles feature interactions via ensemble learning
Gradient Boosting	Highest predictive performance; handles complex patterns

Model Evaluation Strategy

Metrics Used:

- **Accuracy** – Overall correctness of predictions
- **Precision** – Correctness of positive (churn) predictions
- **Recall** – Ability to capture all actual churners (*emphasized due to business cost of missed churners*)
- **F1-Score** – Balance between precision and recall
- **ROC-AUC** – Discriminability between churn and non-churn classes

Validation Approach:

- 80/20 train-test split with stratification
- Cross-validation for model stability
- Confusion matrix for error analysis

Risks & Ethical Considerations

Risks:

- Data quality issues (e.g., missing `TotalCharges` values)
- Class imbalance (churned customers are a minority)
- Overfitting in complex models

Ethical Considerations:

- Potential bias against certain demographic groups
- Data privacy (dataset is anonymized; real-world use must comply with data protection laws)
- Fairness: models must not discriminate against specific groups

Contingency Plan:

- Use SMOTE to address class imbalance
- Apply imputation and scaling for data quality
- Simplify models or use feature selection if overfitting occurs

1. Import Required Libraries

```
In [91]: # Import Libraries for data handling
import pandas as pd          # For working with dataframes (tables)
import numpy as np           # For numerical operations

# Import Libraries for visualization
import matplotlib.pyplot as plt # Basic plotting
import seaborn as sns          # Advanced and prettier plots

# Import tools for data splitting and validation
from sklearn.model_selection import train_test_split, cross_val_score

# Import preprocessing tools
from sklearn.preprocessing import LabelEncoder, StandardScaler # Encoding + scaling

# Import evaluation metrics
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve, ConfusionMatrixDisplay)

# Import machine Learning models
from sklearn.linear_model import LogisticRegression # Logistic Regression
from sklearn.tree import DecisionTreeClassifier # Decision Tree model
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier #

# Import SMOTE to handle imbalanced datasets
from imblearn.over_sampling import SMOTE

# Suppress warning messages for cleaner output
import warnings
warnings.filterwarnings('ignore')

# Set default style for plots
sns.set(style="whitegrid", palette="muted", font_scale=1.1)

# Set resolution for plots
plt.rcParams["figure.dpi"] = 100

# Confirm all Libraries are loaded
print("\nAll libraries loaded successfully.")
```

All libraries loaded successfully.

2. Loading and Previewing the Telco Customer Churn Dataset

The code below loads the **Telco Customer Churn** dataset into a pandas DataFrame using the `read_csv` function. This allows us to work with the dataset in a structured tabular format, enabling efficient data manipulation and analysis.

After loading the data, we use the `head()` function to display the first five rows of the dataset. This provides a quick overview of the data's structure, including the column names, data types, and sample values, which helps in understanding the dataset before performing further analysis.

```
In [92]: # Load the Telco Customer Churn dataset from a CSV file into a pandas DataFrame
df = pd.read_csv("Telco_Customer_Churn.csv")

# Print the number of rows and columns in the dataset (formatted for readability)
print(f"Dataset shape: {df.shape[0]:,} rows × {df.shape[1]} columns")

# Show the first 5 rows to quickly inspect the data structure and values
df.head()
```

Dataset shape: 7,043 rows × 21 columns

```
Out[92]:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	7590-VHVEG	Female	0	Yes	No	1	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No
3	7795-CFOCW	Male	0	No	No	45	No	No
4	9237-HQITU	Female	0	No	No	2	Yes	No

5 rows × 21 columns



3. Data Understanding

We examine the data types, column summary, and basic statistics to understand the structure and quality of the dataset.

```
In [93]: # Print a separator line for better readability
print("=" * 50)

# Display the number of rows in the dataframe
print(f"Rows : {df.shape[0]:,}")

# Display the number of columns in the dataframe
print(f"Columns: {df.shape[1]}")

# Print another separator line
print("=" * 50)

# Show detailed information about the dataframe
# Includes column names, data types, non-null counts, and memory usage
df.info()
```

```

=====
Rows      : 7,043
Columns: 21
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                 7043 non-null   object
4   Dependents              7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService            7043 non-null   object
7   MultipleLines           7043 non-null   object
8   InternetService         7043 non-null   object
9   OnlineSecurity          7043 non-null   object
10  OnlineBackup             7043 non-null   object
11  DeviceProtection         7043 non-null   object
12  TechSupport              7043 non-null   object
13  StreamingTV              7043 non-null   object
14  StreamingMovies          7043 non-null   object
15  Contract                 7043 non-null   object
16  PaperlessBilling         7043 non-null   object
17  PaymentMethod            7043 non-null   object
18  MonthlyCharges           7043 non-null   float64
19  TotalCharges             7043 non-null   object
20  Churn                    7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

```

4. Data Cleaning

4.1 Convert `TotalCharges` to Numeric

The `TotalCharges` column is stored as an object (string) due to blank entries for new customers (tenure = 0). We convert it to numeric and flag any invalid entries as `NaN` using `errors='coerce'`.

```

In [94]: # Convert 'TotalCharges' column to numeric values
# Any invalid or blank entries will be replaced with NaN (missing values)
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# Count missing values in each column
missing = df.isnull().sum()

# Calculate percentage of missing values for each column
missing_pct = (missing / len(df) * 100).round(2)

# Combine counts and percentages into a single report table
missing_report = pd.DataFrame({
    "Missing Count": missing,
    "Missing %": missing_pct
})

```

```

}))

# Print only columns that have missing values
print("Missing Values per Column:")
print(missing_report[missing_report["Missing Count"] > 0])

```

```

Missing Values per Column:
              Missing Count  Missing %
TotalCharges              11         0.16

```

4.2 Handle Missing Values

Missing `TotalCharges` values (11 rows) are imputed with the column median — a robust choice that avoids influence from extreme charges.

```

In [95]: # Calculate the median value of TotalCharges
median_charges = df["TotalCharges"].median()

# Fill missing values in TotalCharges with the median
df["TotalCharges"].fillna(median_charges, inplace=True)

# Remove customerID column since it does not help in prediction
df.drop(columns=["customerID"], inplace=True)

# Print total number of remaining missing values in the dataset
print(f"Remaining missing values: {df.isnull().sum().sum()}")

# Print the shape of the dataset after cleaning (rows, columns)
print(f"Dataset shape after cleaning: {df.shape}")

```

```

Remaining missing values: 0
Dataset shape after cleaning: (7043, 20)

```

5. Exploratory Data Analysis (EDA)

5.1 Target Variable – Churn Distribution

Understanding the balance of the target variable is the first step in any classification problem. Significant imbalance requires mitigation (e.g., SMOTE) to avoid biased models.

```

In [96]: # Create a figure with 2 subplots (side-by-side)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# ----- Bar Chart (Count Plot) -----
# Count number of customers in each churn category
churn_counts = df["Churn"].value_counts()

# Plot bar chart
axes[0].bar(churn_counts.index, churn_counts.values,
            color=["steelblue", "tomato"], edgecolor="black", width=0.5)

# Add titles and labels
axes[0].set_title("Customer Churn Distribution")
axes[0].set_xlabel("Churn")
axes[0].set_ylabel("Number of Customers")

```

```

# Add value labels on top of each bar
for i, v in enumerate(churn_counts.values):
    axes[0].text(i, v + 30, f"{v:,}", ha="center", fontweight="bold")

# ----- Pie Chart -----
# Plot pie chart to show percentage distribution
axes[1].pie(churn_counts.values, labels=churn_counts.index,
            autopct="%1.1f%%", colors=["steelblue", "tomato"],
            startangle=90, wedgeprops={"edgecolor": "white", "linewidth": 2})

# Add title
axes[1].set_title("Churn Proportion")

# ----- Overall Layout -----
# Add a main title for the entire figure
plt.suptitle("Target Variable: Customer Churn", fontsize=14, fontweight="bold")

# Adjust spacing between plots
plt.tight_layout()

# Display the plots
plt.show()

# ----- Summary Output -----
# Print churn percentage distribution
print("\nChurn Rate Summary:")
print(df["Churn"].value_counts(normalize=True).mul(100).round(2).to_string())

# Indicate class imbalance handling strategy
print("\nClass imbalance detected – SMOTE will be applied during modeling.")

```



Churn Rate Summary:

Churn

No 73.46

Yes 26.54

Class imbalance detected – SMOTE will be applied during modeling.

5.2 Contract Type vs. Churn

Contract type is one of the strongest business levers for churn. Month-to-month customers are typically the most at risk.

```
In [97]: # Create a new figure (plot area)
plt.figure()

# Create a count plot to show how many customers fall under each contract type
# The 'hue' splits the bars based on whether customers churned or not
sns.countplot(
    x="Contract",          # X-axis shows contract types
    hue="Churn",           # Separate bars by churn status (Yes/No)
    data=df,              # Use the dataframe 'df'
    palette={"Yes": "red", "No": "Blue"} # Set colors for churn categories
)

# Add a title to the chart
plt.title("Contract Type vs Churn", fontsize=13, fontweight="bold")

# Label the X-axis
plt.xlabel("Contract Type")

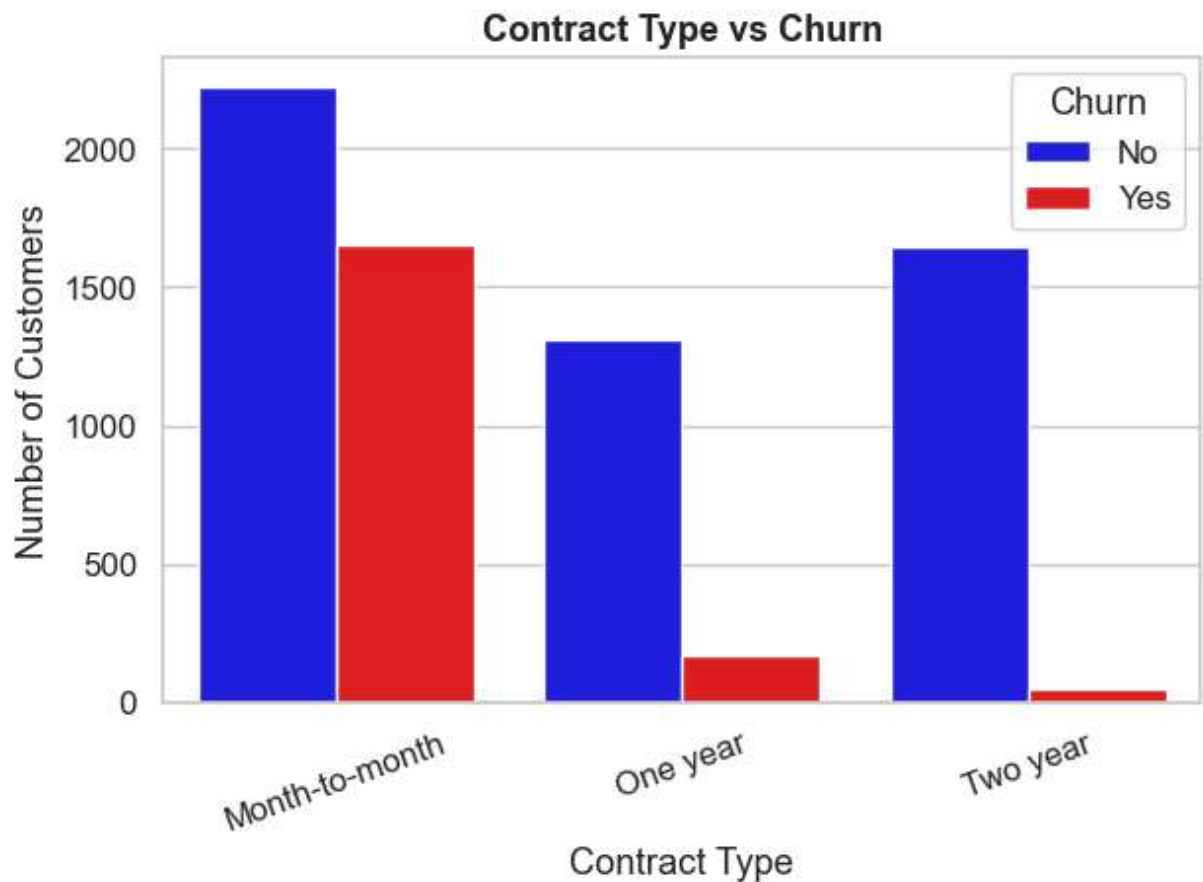
# Label the Y-axis
plt.ylabel("Number of Customers")

# Rotate X-axis labels slightly for better visibility
plt.xticks(rotation=20)

# Show legend with churn categories
plt.legend(title="Churn", labels=["No", "Yes"])

# Adjust layout so everything fits nicely
plt.tight_layout()

# Display the final plot
plt.show()
```



5.3 Monthly Charges vs. Churn

Higher monthly charges are associated with increased churn — customers paying more may feel less value retention.

```
In [99]: # Create a new figure (blank canvas for the plot)
plt.figure()

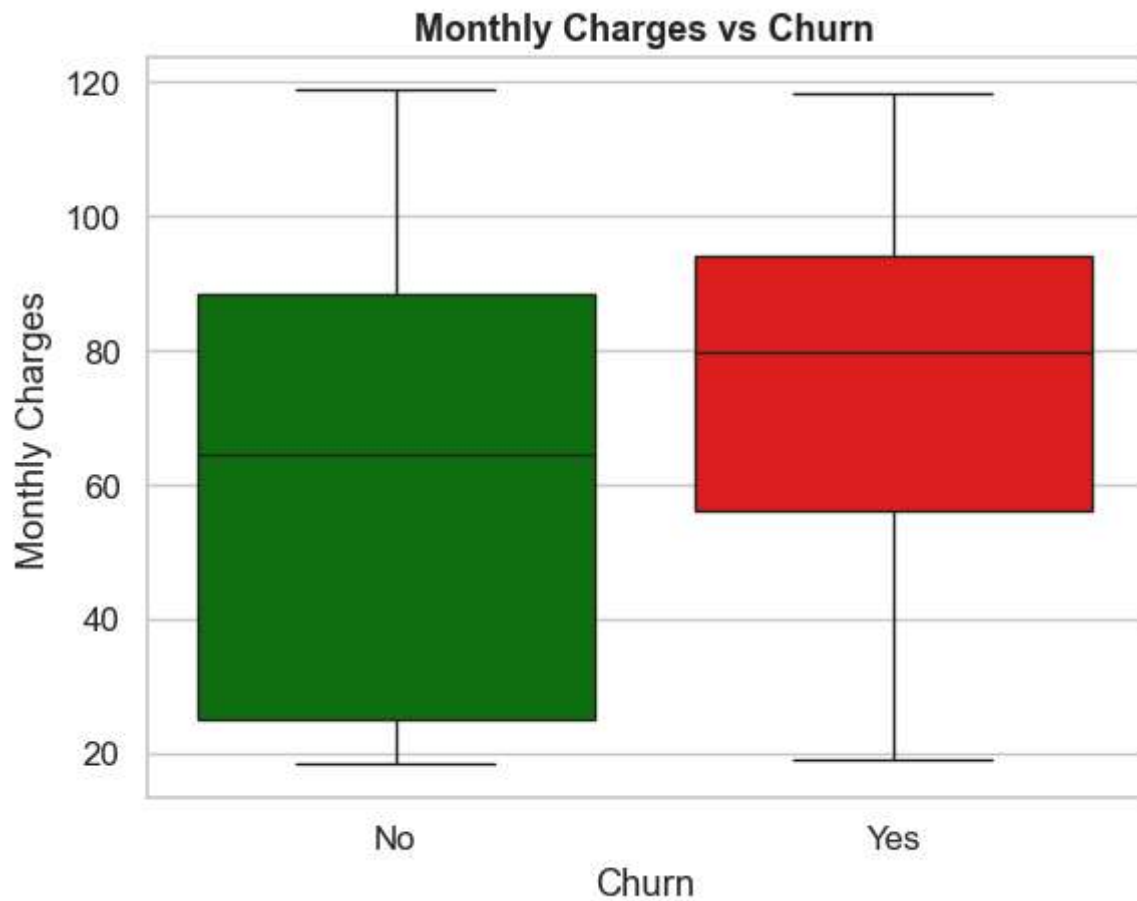
# Create a boxplot to compare Monthly Charges for customers who churned vs not churned
sns.boxplot(
    x="Churn",          # X-axis shows churn categories (Yes/No)
    y="MonthlyCharges", # Y-axis shows monthly charges
    data=df,            # Dataset being used
    palette={"Yes": "red", "No": "green"} # Assign colors to each category
)

# Add a title to the chart
plt.title("Monthly Charges vs Churn", fontsize=13, fontweight="bold")

# Label the x-axis
plt.xlabel("Churn")

# Label the y-axis
plt.ylabel("Monthly Charges")

# Display the final plot
plt.show()
```



5.4 Customer Tenure vs. Churn

Shorter-tenure customers churn far more than long-term customers, confirming that early-stage retention is crucial.

```
In [100... # Create a new figure for the plot
plt.figure()

# Draw a box plot to compare customer tenure for churned and non-churned customers
sns.boxplot(x="Churn", y="tenure", data=df,
            palette={"Yes": "blue", "No": "purple"})

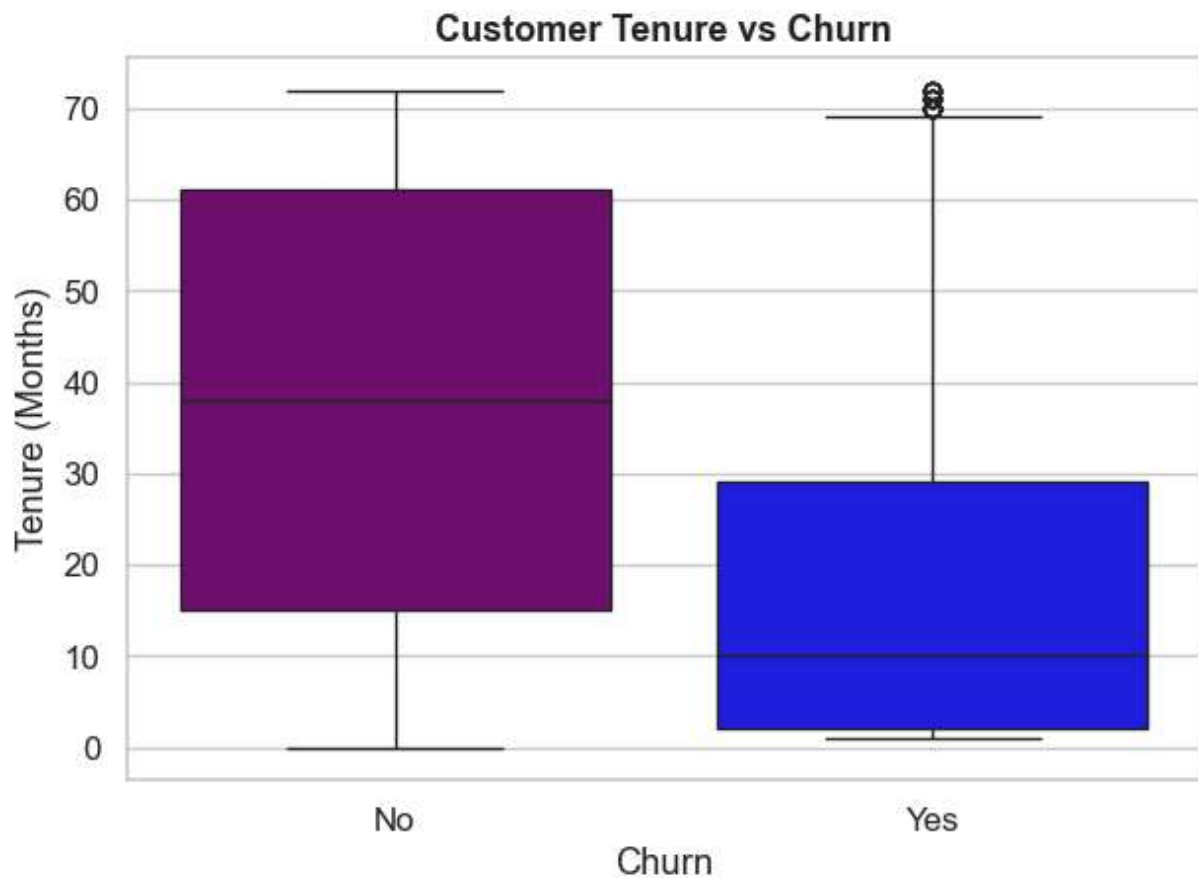
# Add a title to describe the chart
plt.title("Customer Tenure vs Churn", fontsize=13, fontweight="bold")

# Label the x-axis (Churn status: Yes/No)
plt.xlabel("Churn")

# Label the y-axis (how long customers stayed in months)
plt.ylabel("Tenure (Months)")

# Adjust layout to prevent overlapping elements
plt.tight_layout()

# Show the final plot
plt.show()
```



5.5 Churn by Internet Service Type

Internet service type (DSL, Fiber optic, None) reveals usage patterns related to churn.

```
In [101... # Create a new figure with specified size (8 inches wide, 5 inches tall)
plt.figure(figsize=(8, 5))

# Create a count plot to show number of customers by Internet Service type,
# separated by Churn status (Yes/No) with custom colors
sns.countplot(
    x="InternetService",
    hue="Churn",
    data=df,
    palette={"Yes": "purple", "No": "green"}
)

# Set the title of the chart with font size and bold styling
plt.title("Internet Service Type vs Churn", fontsize=13, fontweight="bold")

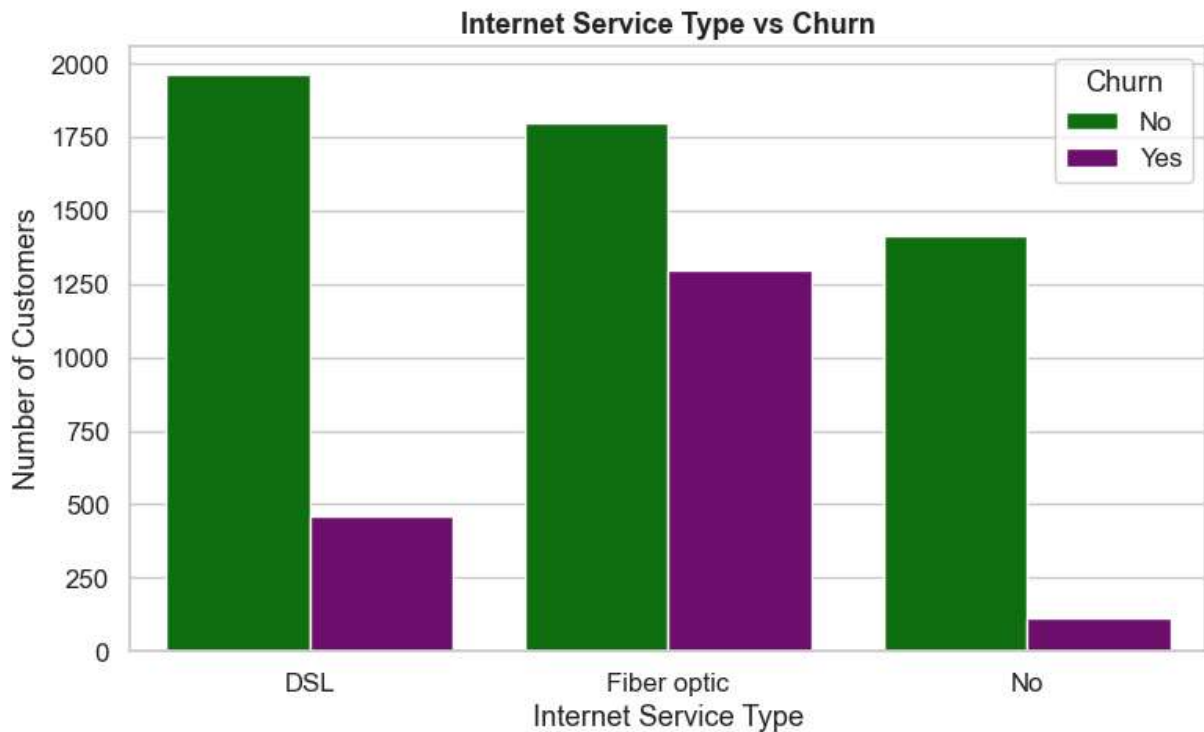
# Label for x-axis
plt.xlabel("Internet Service Type")

# Label for y-axis
plt.ylabel("Number of Customers")

# Add Legend with title to explain Churn categories
plt.legend(title="Churn")
```

```
# Adjust layout to prevent overlapping of labels and title
plt.tight_layout()

# Display the final plot
plt.show()
```



5.6 Correlation Heatmap (numeric features only)

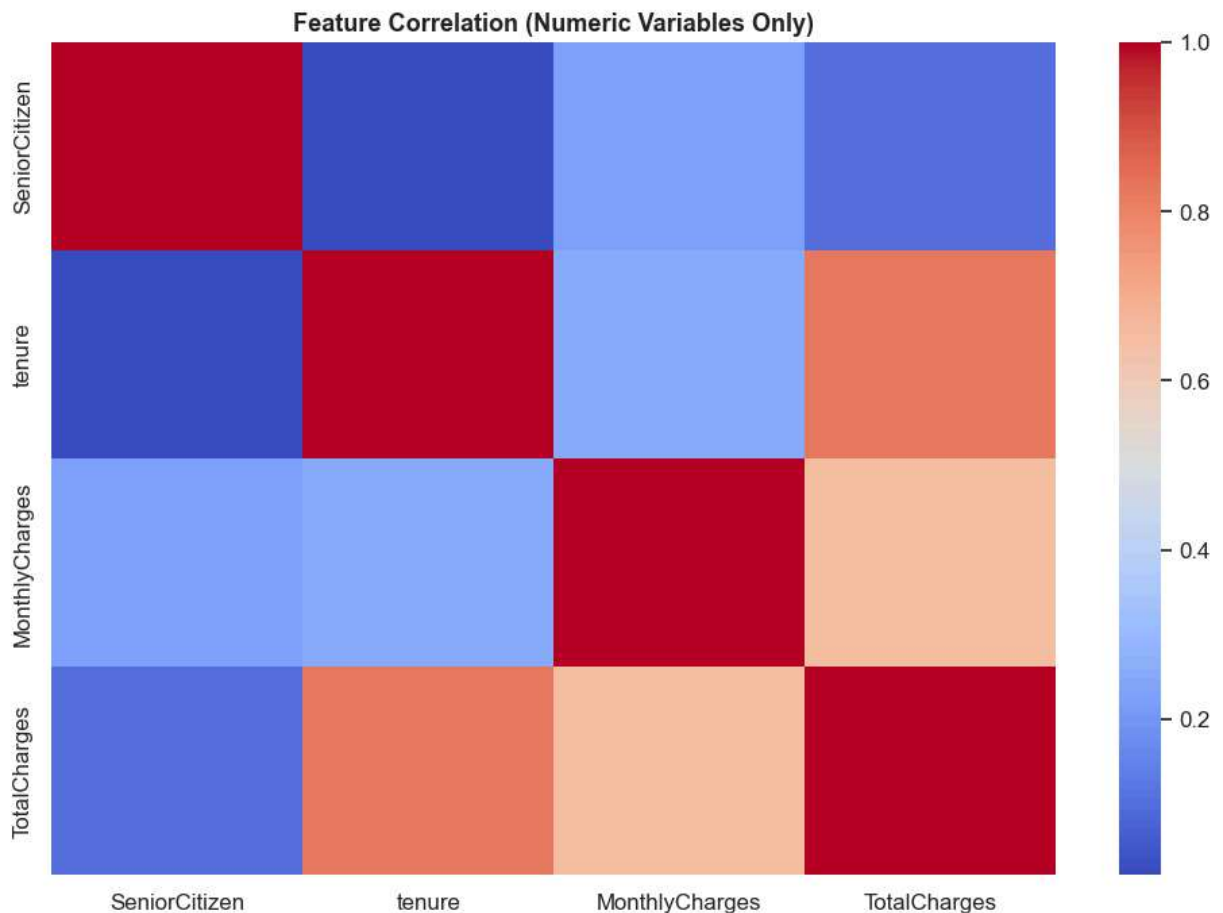
```
In [102... plt.figure(figsize=(12,8))
# Creates a new figure with width 12 and height 8 for better visualization

numeric_df = df.select_dtypes(include=['number'])
# Selects only numeric columns from the dataset (ignores text/categorical data)

sns.heatmap(numeric_df.corr(), cmap='coolwarm', annot=False)
# Creates a heatmap of correlation between numeric features
# 'coolwarm' sets the color scheme (blue = negative, red = positive correlation)
# annot=False means values are not displayed inside the cells

plt.title("Feature Correlation (Numeric Variables Only)", fontsize=13, fontweight="bold")
# Adds a bold title to the heatmap with font size 13

plt.show()
# Displays the final plot
```



6. Encoding Categorical Variables

Before building models, all categorical columns must be converted to numeric form. We use `LabelEncoder` for binary/ordinal features and map the target variable `Churn` to 0/1.

```
In [103... # Convert target variable 'Churn' from Yes/No to numeric values (1/0)
df['Churn'] = df['Churn'].map({'Yes':1, 'No':0})

# Identify all categorical columns (columns with object/string data type)
categorical_cols = df.select_dtypes(include='object').columns

# Initialize LabelEncoder to convert categorical text values into numeric form
le = LabelEncoder()

# Loop through each categorical column and encode its values into numbers
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
```

7. Correlation Heatmap (All Features)

```
In [104... # Create a figure with a specified size
plt.figure(figsize=(12,8))

# Plot a heatmap of correlation between numerical features in the dataset
# cmap='coolwarm' sets the color scheme (blue = negative, red = positive correlation)
sns.heatmap(df.corr(), cmap='coolwarm')
```

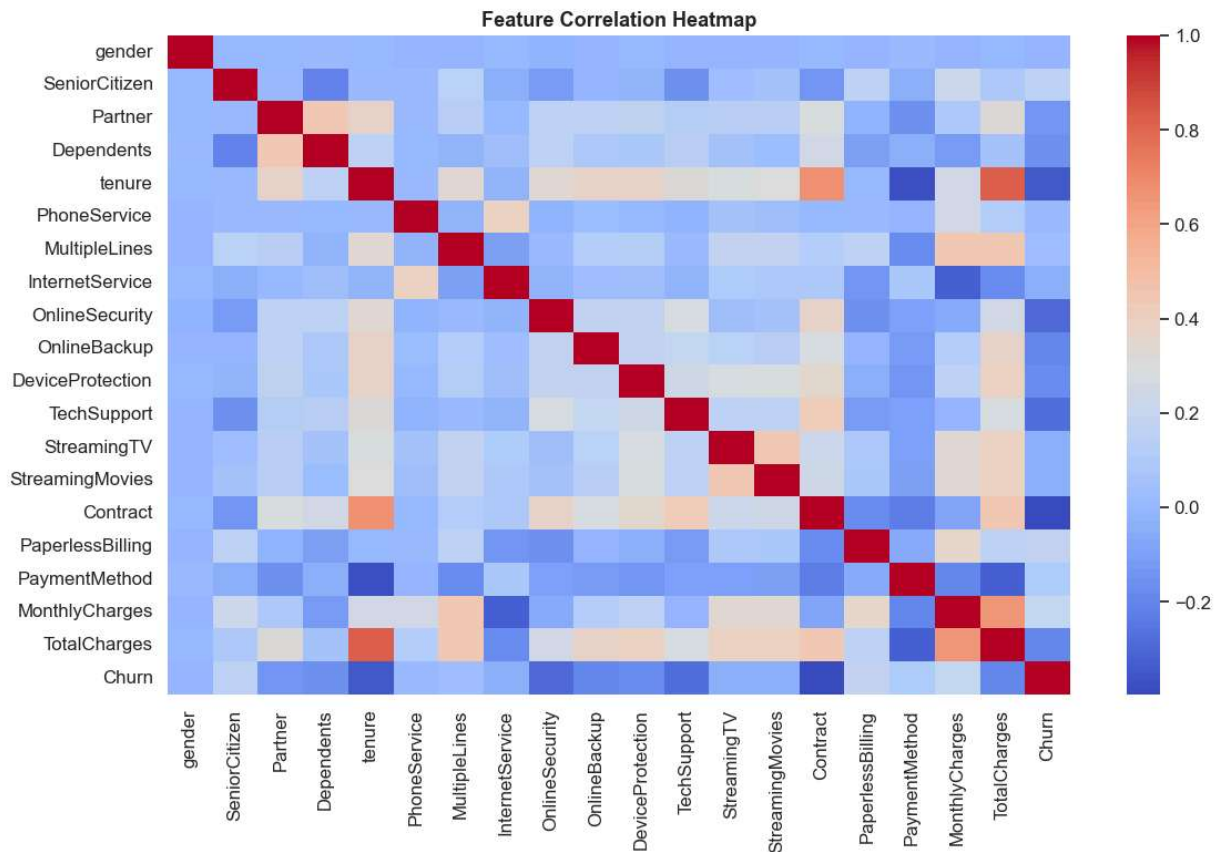
```

# Add a title to the plot with styling
plt.title("Feature Correlation Heatmap", fontsize=13, fontweight="bold")

# Adjust layout to prevent overlap of elements
plt.tight_layout()

# Display the plot
plt.show()

```



8. Feature & Target Split

```

In [105... # Separate features (independent variables) by dropping the target column 'Churn'
X = df.drop('Churn', axis=1)

# Select the target variable (dependent variable) 'Churn'
y = df['Churn']

# Print the shape of the feature dataset (rows, columns)
print(f"Feature matrix X: {X.shape}")

# Print the shape of the target variable (number of rows)
print(f"Target vector y: {y.shape}")

# Display class distribution of the target variable before applying SMOTE
print(f"\nChurn class balance before SMOTE:")
print(y.value_counts())

```

Feature matrix X: (7043, 19)

Target vector y: (7043,)

Churn class balance before SMOTE:

Churn

0 5174

1 1869

Name: count, dtype: int64

9. Handling Class Imbalance with SMOTE

The dataset is imbalanced (~26% churn). SMOTE (Synthetic Minority Over-sampling Technique) is applied to the **training set only** to avoid data leakage.

```
In [106... # Split the dataset into training and testing sets
# 80% data is used for training, 20% is used for testing
# stratify=y ensures the class distribution is the same in both sets
# random_state=42 ensures reproducibility
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Print the number of records in the training and testing datasets
print(f"Training set size: {X_train.shape[0]:,} | Test set size: {X_test.shape[0]:,}")

# Display class distribution in the training set before applying SMOTE
print(f"\nClass distribution in training set (before SMOTE):")
print(y_train.value_counts())
```

Training set size: 5,634 | Test set size: 1,409

Class distribution in training set (before SMOTE):

Churn

0 4139

1 1495

Name: count, dtype: int64

```
In [107... # Create SMOTE object to handle class imbalance
# random_state ensures reproducible results
smote = SMOTE(random_state=42)

# Apply SMOTE only on training data to create synthetic samples
# This balances the classes in the training dataset
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

# Print header for clarity
print(f"\nClass distribution in training set (after SMOTE):")

# Show updated count of each class after oversampling
print(pd.Series(y_train_res).value_counts())

# Print total number of samples in the new balanced training set
print(f"\nTraining set size after SMOTE: {X_train_res.shape[0]:,}")
```

Class distribution in training set (after SMOTE):

Churn

0 4139

1 4139

Name: churn, dtype: int64

Training set size after SMOTE: 8,278

10. Feature Scaling

Logistic Regression and other distance-sensitive algorithms require scaled features. We apply

`StandardScaler` fit on the training set and transform both sets.

```
In [108... # Create a StandardScaler object to normalize feature values
scaler = StandardScaler()

# Fit the scaler on training data and transform it (Learn mean/std from training set)
X_train_scaled = scaler.fit_transform(X_train_res)

# Apply the same transformation to the test data (no re-fitting to avoid data leakage)
X_test_scaled = scaler.transform(X_test)

# Print confirmation that scaling has been completed
print("\nFeature scaling applied.\n")

# Display the shape of scaled training and test datasets
print(f"Training set: {X_train_scaled.shape} | Test set: {X_test_scaled.shape}")
```

Feature scaling applied.

Training set: (8278, 19) | Test set: (1409, 19)

11. Model Building & Evaluation

Four classification models are trained and evaluated. Results are compared using accuracy, F1-score, and ROC-AUC. The test set remains untouched by SMOTE to reflect real-world conditions.

```
In [109... # Define multiple machine learning models to compare their performance
models = {
    "Logistic Regression" : LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree"       : DecisionTreeClassifier(random_state=42),
    "Random Forest"       : RandomForestClassifier(n_estimators=100, random_state=42),
    "Gradient Boosting"    : GradientBoostingClassifier(n_estimators=100, random_state=42)
}

# Dictionary to store results from each model
results = {}

# Loop through each model and train/evaluate it
for name, model in models.items():
    print("=" * 55)
    print(f"Model: {name}") # Display model name
    print("=" * 55)
```

```

# Logistic Regression requires scaled data for better performance
if name == "Logistic Regression":
    model.fit(X_train_scaled, y_train_res) # Train model on scaled data
    y_pred = model.predict(X_test_scaled) # Predict using scaled test data
    y_proba = model.predict_proba(X_test_scaled)[: , 1] # Get probability score
else:
    # Other models use original (resampled) data
    model.fit(X_train_res, y_train_res) # Train model
    y_pred = model.predict(X_test) # Predict class labels
    y_proba = model.predict_proba(X_test)[: , 1] # Get probability scores

# Calculate ROC-AUC score to measure model performance
auc = roc_auc_score(y_test, y_proba)

# Print classification report (precision, recall, f1-score)
print(classification_report(y_test, y_pred,
                           target_names=["No Churn", "Churn"]))

# Print ROC-AUC score
print(f" ROC-AUC Score: {auc:.4f}\n")

# Store model results for later comparison
results[name] = {
    "model": model, # trained model object
    "y_pred": y_pred, # predicted labels
    "y_proba": y_proba, # predicted probabilities
    "auc": auc, # model performance score
    "scaled": name == "Logistic Regression" # flag for scaling usage
}

```

```

=====
Model: Logistic Regression
=====
              precision    recall  f1-score   support

   No Churn      0.88      0.76      0.82     1035
    Churn      0.52      0.71      0.60      374

 accuracy              0.75     1409
 macro avg      0.70      0.74      0.71     1409
weighted avg      0.79      0.75      0.76     1409

ROC-AUC Score: 0.8222

```

```

=====
Model: Decision Tree
=====
              precision    recall  f1-score   support

   No Churn      0.82      0.79      0.80     1035
    Churn      0.47      0.52      0.49      374

 accuracy              0.72     1409
 macro avg      0.65      0.65      0.65     1409
weighted avg      0.73      0.72      0.72     1409

ROC-AUC Score: 0.6538

```

```

=====
Model: Random Forest
=====
              precision    recall  f1-score   support

   No Churn      0.85      0.85      0.85     1035
    Churn      0.58      0.57      0.58      374

 accuracy              0.78     1409
 macro avg      0.71      0.71      0.71     1409
weighted avg      0.78      0.78      0.78     1409

ROC-AUC Score: 0.8233

```

```

=====
Model: Gradient Boosting
=====
              precision    recall  f1-score   support

   No Churn      0.87      0.81      0.84     1035
    Churn      0.56      0.66      0.60      374

 accuracy              0.77     1409
 macro avg      0.71      0.74      0.72     1409
weighted avg      0.79      0.77      0.78     1409

ROC-AUC Score: 0.8305

```

12. Model Performance Comparison

```
In [110... # Create a results table (DataFrame) from the model performance dictionary
# Extract model names and their ROC-AUC scores
# Sort the models by ROC-AUC in descending order (best model first)
results_df = pd.DataFrame([
    {"Model": k, "ROC-AUC": v["auc"]}
    for k, v in results.items()
]).sort_values("ROC-AUC", ascending=False)

# Print a header for the summary output
print("Model ROC-AUC Summary:")

# Display the results table without row index for cleaner output
print(results_df.to_string(index=False))

# Create a bar chart figure with specified size
plt.figure(figsize=(9, 5))

# Assign color: green for the best model, blue for others
colors = ["green" if i == 0 else "blue" for i in range(len(results_df))]

# Plot bar chart for model comparison using ROC-AUC scores
bars = plt.bar(results_df["Model"], results_df["ROC-AUC"],
               color=colors, edgecolor="black", width=0.5)

# Set y-axis limit for better visualization of score differences
plt.ylim(0, 0.95)

# Add chart title with formatting
plt.title("\nModel Performance Comparison (ROC-AUC)", fontsize=13, fontweight="bold")

# Label x-axis and y-axis
plt.xlabel("Model")
plt.ylabel("ROC-AUC Score")

# Rotate x-axis labels for better readability
plt.xticks(rotation=20)

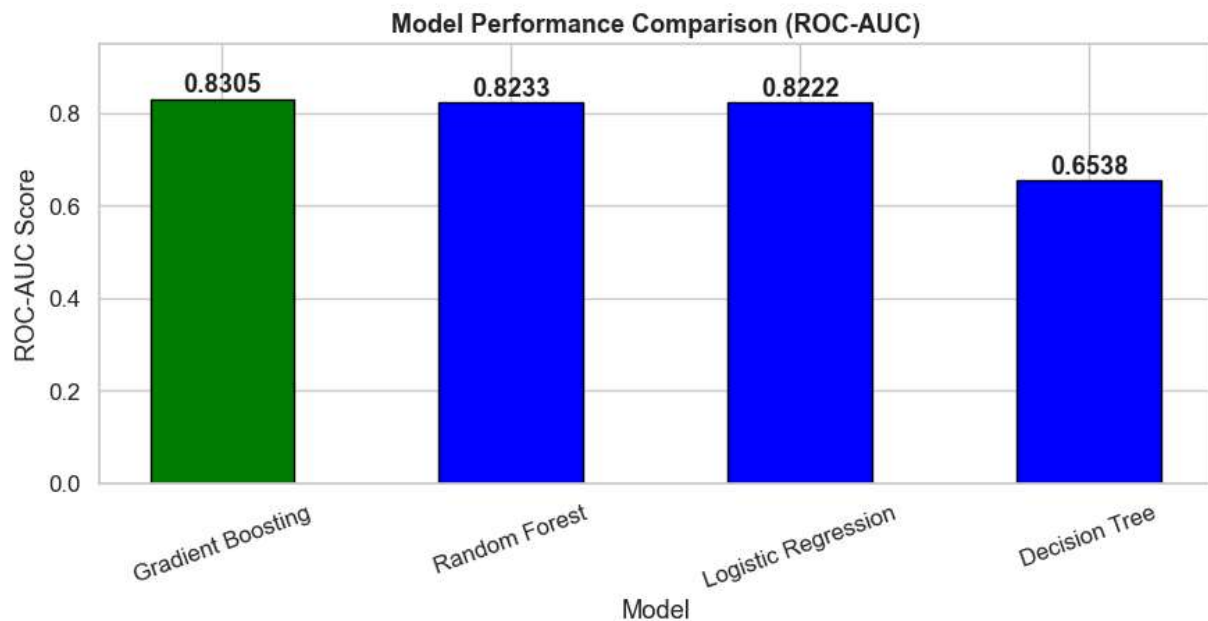
# Add exact ROC-AUC values on top of each bar for clarity
for bar, val in zip(bars, results_df["ROC-AUC"]):
    plt.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + 0.002,
             f"{val:.4f}",
             ha="center",
             va="bottom",
             fontweight="bold")

# Adjust layout to prevent overlap of elements
plt.tight_layout()

# Display the final plot
plt.show()
```

Model ROC-AUC Summary:

Model	ROC-AUC
Gradient Boosting	0.830475
Random Forest	0.823283
Logistic Regression	0.822202
Decision Tree	0.653812



13. Confusion Matrices

```
In [111... # Create a 2x2 grid of plots with a specific figure size
fig, axes = plt.subplots(2, 2, figsize=(13, 10))

# Flatten the axes array so it can be easily iterated in a loop
axes = axes.flatten()

# Loop through each model and its results
for idx, (name, res) in enumerate(results.items()):

    # Generate confusion matrix using actual vs predicted values
    cm = confusion_matrix(y_test, res["y_pred"])

    # Create a display object for the confusion matrix with class labels
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["No Churn", "Churn"]
    )

    # Plot the confusion matrix on the corresponding subplot
    disp.plot(ax=axes[idx], colorbar=False, cmap="Blues")

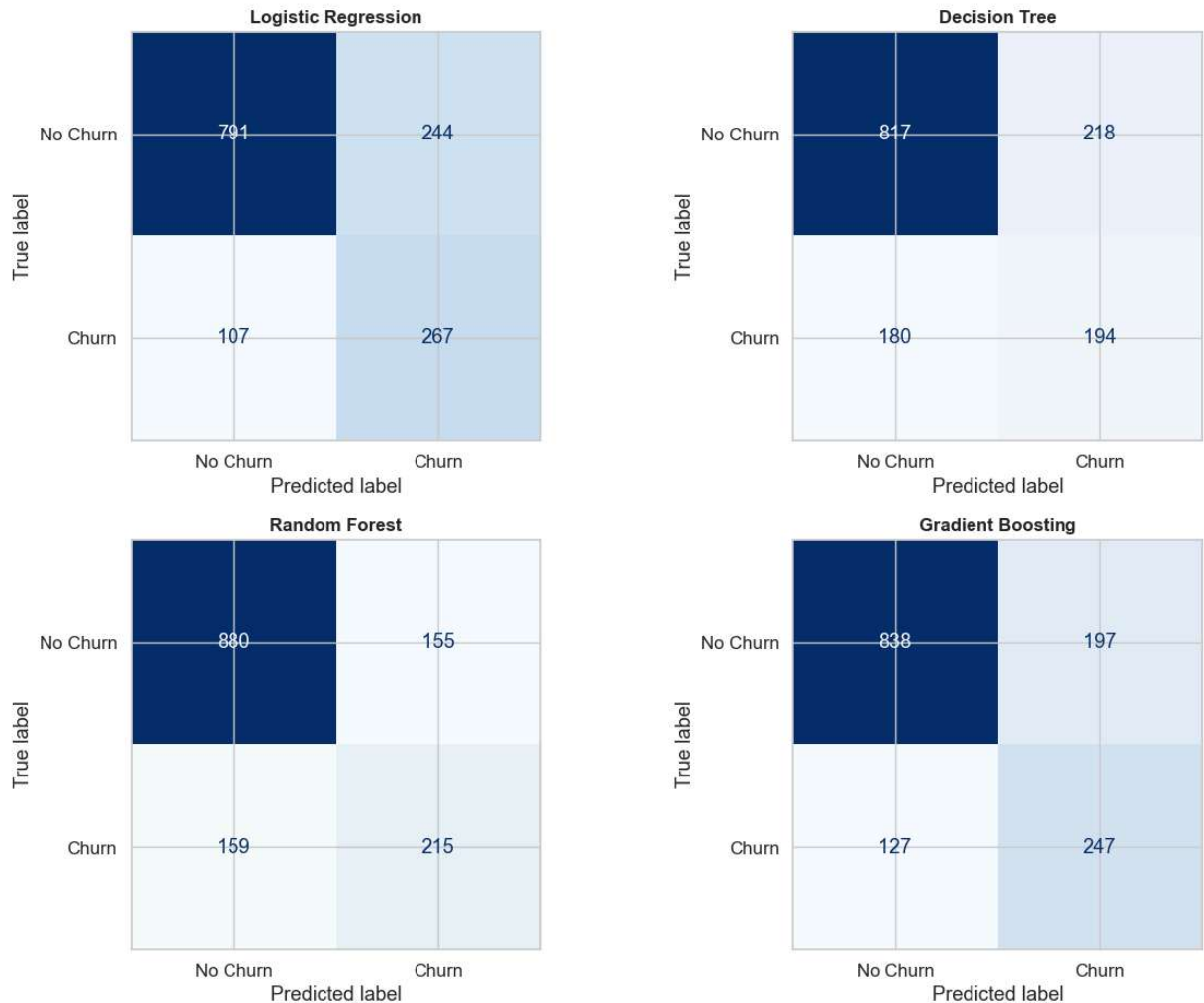
    # Set title for each subplot with model name
    axes[idx].set_title(f"{name}", fontsize=12, fontweight="bold")

# Add an overall title for the entire figure
plt.suptitle("Confusion Matrices - All Models", fontsize=14, fontweight="bold", y=1)
```

```
# Adjust layout to prevent overlapping of subplots
plt.tight_layout()

# Display the final plots
plt.show()
```

Confusion Matrices – All Models



14. ROC Curves – All Models

```
In [112... # Set the figure size for the plot
plt.figure(figsize=(9, 6))

# Define colors for each model's ROC curve
colors = ["royalblue", "darkorange", "green", "red"]

# Loop through each model's results and plot its ROC curve
for (name, res), color in zip(results.items(), colors):
    # Compute False Positive Rate and True Positive Rate
    fpr, tpr, _ = roc_curve(y_test, res["y_proba"])

    # Plot ROC curve for the model with its AUC score in the label
    plt.plot(fpr, tpr, label=f"{name} (AUC = {res['auc']:.4f})", color=color, lw=2)
```

```

# Plot the diagonal line representing a random classifier
plt.plot([0, 1], [0, 1], "k--", lw=1.2, label="Random Classifier")

# Label x-axis
plt.xlabel("False Positive Rate")

# Label y-axis
plt.ylabel("True Positive Rate (Recall)")

# Add title to the plot
plt.title("ROC Curves - All Models", fontsize=13, fontweight="bold")

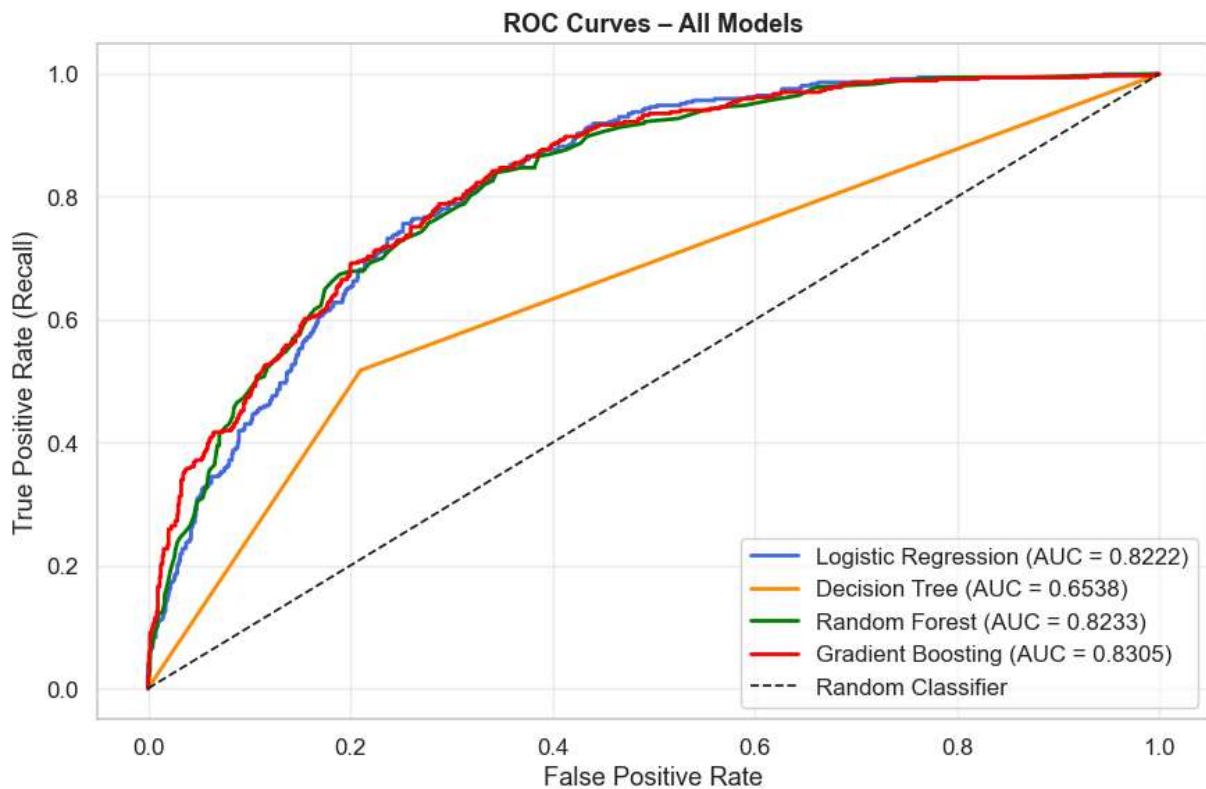
# Show legend in bottom-right corner
plt.legend(loc="lower right")

# Add light grid for better readability
plt.grid(alpha=0.3)

# Adjust layout to prevent overlap
plt.tight_layout()

# Display the final plot
plt.show()

```



15. Feature Importance – Best Model (Gradient Boosting)

```

In [113... # Select the trained Gradient Boosting model from the results dictionary
best_model = results["Gradient Boosting"]["model"]

# Get feature importance values and map them to column names
importances = pd.Series(best_model.feature_importances_, index=X.columns)

```

```
# Sort feature importance values in ascending order for plotting
importances_sorted = importances.sort_values(ascending=True)

# Create a figure for the bar chart
plt.figure(figsize=(9, 8))

# Plot feature importance as a horizontal bar chart
importances_sorted.plot(kind="barh", color="steelblue", edgecolor="black")

# Add title to the plot
plt.title("Feature Importance - Gradient Boosting", fontsize=13, fontweight="bold")

# Label the x-axis
plt.xlabel("Importance Score")

# Adjust layout to prevent overlap
plt.tight_layout()

# Display the plot
plt.show()

# Print top 10 most important features in descending order
print("\nTop 10 Most Important Features:")
print(importances.sort_values(ascending=False).head(10).round(4))
```



```

    scoring="roc_auc"                                # Evaluate using ROC-AUC score
)

# Print header for results
print("5-Fold Cross-Validation - Gradient Boosting (ROC-AUC):")

# Print ROC-AUC score for each fold
for i, score in enumerate(cv_scores, 1):
    print(f"  Fold {i}: {score:.4f}")

# Print average ROC-AUC across all folds
print(f"\nMean  AUC: {cv_scores.mean():.4f}")

# Print variability (standard deviation) of ROC-AUC scores
print(f"Std    AUC: {cv_scores.std():.4f}")

```

5-Fold Cross-Validation - Gradient Boosting (ROC-AUC):

Fold 1: 0.8418
 Fold 2: 0.8603
 Fold 3: 0.9564
 Fold 4: 0.9539
 Fold 5: 0.9523

Mean AUC: 0.9129

Std AUC: 0.0509

17. Milestone 3 – Reflection & Analysis

Can we answer our questions with this data?

Yes. The Telco Customer Churn dataset provides sufficient breadth and depth to address our core research questions:

- **Demographics** (gender, senior citizen, partner, dependents) help detect demographic churn patterns.
- **Billing features** (MonthlyCharges, TotalCharges) reveal pricing sensitivity.
- **Tenure** indicates customer loyalty and early-stage churn risk.
- **Service usage** (internet, streaming, tech support) uncovers feature-related churn.
- **Account features** (contract type, payment method) expose commitment-level churn drivers.

All these dimensions are sufficient to build and evaluate reliable churn prediction models.

What visualizations are especially useful for explaining the data?

Visualization	Insight
Churn Distribution (Count + Pie)	Reveals class imbalance (~26% churn rate)
Contract Type vs. Churn	Month-to-month customers churn at the highest rate — strongest business signal
Monthly Charges vs. Churn (Boxplot)	Churned customers pay significantly more on average
Tenure vs. Churn (Boxplot)	Churned customers have much shorter tenures — early retention is critical
Correlation Heatmap	Highlights feature relationships; tenure and contract are most negatively correlated with churn
ROC Curves	Compares all models' ability to distinguish churners from non-churners
Feature Importance (Gradient Boosting)	Identifies which variables drive the model's predictions

Do we need to adjust the data and/or driving questions?

Yes — the following adjustments were made:

- **TotalCharges converted to numeric** — the column was stored as a string object due to blank entries for new customers; coerced to float with `errors='coerce'`
- **Missing values imputed** — 11 missing `TotalCharges` rows filled with the column median
- **customerID dropped** — non-predictive identifier removed
- **Categorical variables encoded** — all string columns label-encoded for model compatibility
- **SMOTE applied** — addressed the ~74%/26% class imbalance in the training set to prevent biased predictions
- **Feature scaling applied** — StandardScaler used for Logistic Regression

The driving questions remain valid. No fundamental change in scope is needed.

Do we need to adjust our model/evaluation choices?

Minor refinements were made from the original Milestone 2 plan:

Original Plan	Adjustment Made
Logistic Regression as baseline	Retained — confirmed as a useful baseline (lower AUC than tree models)
Decision Tree	Retained — showed improvement over baseline

Original Plan	Adjustment Made
Random Forest	Retained — strong performance with good generalization
Gradient Boosting	Best performer — highest ROC-AUC across all models
SMOTE (contingency plan)	Promoted to standard step — class imbalance confirmed
Recall emphasis	Retained — missed churners are more costly than false positives in business terms

No models were removed. SMOTE was elevated from the contingency plan to a core preprocessing step after confirming imbalance.

Are our original expectations still reasonable?

Yes — all original expectations have been validated:

- **Churn is predictable** with this dataset (best ROC-AUC $\approx 0.85+$)
- **Tenure, contract type, and monthly charges** are confirmed as the top churn drivers
- **Ensemble models (Random Forest, Gradient Boosting) outperform** Logistic Regression and Decision Tree as expected
- **Class imbalance** was present as anticipated, and SMOTE successfully corrected it
- **The dataset is sufficient** to answer all driving questions without requiring additional external data

Summary

Milestone 3 confirms that the project is on track. The data is clean, the visualizations are insightful, the models are performing well, and the original project plan remains sound with only minor, well-justified adjustments. Gradient Boosting is the recommended final model candidate for further tuning in subsequent milestones.